PYTHON

Every Python Function a Data Analyst Needs at ₹15-20 LPA

The complete list of 24 Python and Pandas functions expected from a mid-level data analyst, grouped into 6 jobs, with the code for each. Ends with a self-check.

INSIDE THIS PDF · 9 PAGES

- The 6 jobs, 24 functions
- 1. Load and inspect
- 2. Clean
- 3. Filter and transform
- 4. Summarise
- 5. Combine and reshape
- 6. Dates and automation

@thedataguy16

The 6 jobs, 24 functions

At ₹15-20 LPA you are expected to handle messy, large data on your own and automate repeat work.

- **1. Load and inspect**
  4 functions
- **2. Clean**
  4 functions
- **3. Filter and transform**
  4 functions
- **4. Summarise**
  4 functions
- **5. Combine and reshape**
  4 functions
- **6. Dates and automation**
  4 functions

Score yourself on the last page. 20+ means your Python matches this salary band.

@thedataguy16

1. Load and inspect

The first 5 minutes with any dataset.

read_csv / read_excel

Set types and dates while loading.

```
pd.read_csv("f.csv", parse_dates=["date"],
             usecols=["id", "date", "amount"])
```

info

Types and non-null counts in one view.

```
df.info()
```

describe

Quick stats, including percentiles.

```
df.describe(percentiles=[.5, .9, .99])
```

value_counts

Frequency and share of each value.

```
df["city"].value_counts(normalize=True)
```

@thedataguy16

2. Clean

Real data is always dirty. These four handle most problems.

isna + fillna

Find and fill missing values.

```
df["discount"] = df["discount"].fillna(0)
```

drop_duplicates

Keep the latest row per key.

```
df.sort_values("updated").drop_duplicates(
    "order_id", keep="last")
```

astype / to_numeric

Fix types. errors="coerce" turns bad values into NaN.

```
df["amt"] = pd.to_numeric(df["amt"],
errors="coerce")
```

str methods

Clean text in one line.

```
df["city"] = df["city"].str.strip().str.title()
```

@thedataguy16

3. Filter and transform

Select and create columns without loops.

loc

Filter rows and pick columns, or update values.

```
df.loc[df["amount"] < 0, "amount"] = 0
```

query

Readable multi-condition filters.

```
df.query("region == 'North' and amount > 5000")
```

np.where

If-else on a whole column.

```
df["size"] = np.where(df["amount"] > 10000,
                      "Large", "Small")
```

assign

Add several columns in one chain.

```
df = df.assign(tax=df.amount * 0.18,
               net=lambda d: d.amount - d.tax)
```

@thedataguy16

4. Summarise

These answer most business questions.

- **groupby + agg**

  Named aggregations.

```
df.groupby("region").agg(rev=("amount", "sum"),
    orders=("order_id", "nunique"))
```

- **transform**

  Group value back on every row, for share of group.

```
df["share"] = df.amount / df.groupby(
    "region").amount.transform("sum")
```

- **rank**

  Rank within groups.

```
df["rk"] = df.groupby("dept").salary.rank(
    ascending=False, method="dense")
```

- **nlargest**

  Top N quickly.

```
df.nlargest(10, "amount")
```

@thedataguy16

5. Combine and reshape

Joining tables and building report views.

merge

SQL joins. validate catches duplicate keys.

```
orders.merge(cust, on="cust_id", how="left",
             validate="many_to_one")
```

concat

Stack monthly files.

```
pd.concat([jan, feb, mar], ignore_index=True)
```

pivot_table

Excel-style pivot.

```
df.pivot_table(index="region", columns="month",
    values="amount", aggfunc="sum",
fill_value=0)
```

melt

Wide to long, for month columns.

```
df.melt(id_vars="region", var_name="month",
        value_name="sales")
```

@thedataguy16

6. Dates and automation

This group separates ₹8 LPA from ₹18 LPA analysts.

dt accessor

Month, weekday, period from dates.

```
df["month"] = df["date"].dt.to_period("M")
```

resample

Daily data to weekly or monthly totals.

```
df.set_index("date").amount.resample("W").sum()
```

rolling + pct_change

Moving averages and growth.

```
s.rolling(7).mean(); s.pct_change().mul(100)
```

glob + to_excel

Combine every file in a folder and save the report.

```
files = glob.glob("data/*.csv")
pd.concat(map(pd.read_csv, files)).to_excel(
    "report.xlsx", index=False)
```

@thedataguy16

Score yourself

Count how many of the 24 you can use without searching.

- **20-24**

  Your Python matches ₹15-20 LPA roles. Show it with one automation project.

- **14-19**

  Focus on groups 4-6: transform, merge validate, resample, automation.

- **Below 14**

  Get groups 1-3 solid first. Practise on one messy real dataset daily.

Interview proof: "I automated the weekly report: 12 files merged, cleaned and summarised in one script, saving 5 hours a week." That one line is worth more than the whole list.

@thedataguy16